

# Machine Learning - BSMALEA1KU

Marta Zuzanna Richert – mazr@itu.dk  
Laura Cabral Costa Ascensão Tiago – lcab@itu.dk  
Youssef Alla Eldin Salem – yoal@itu.dk

December 21, 2025

## Abstract

This project, conducted in partnership with Tryg's Data Science group, investigates machine learning methodologies for predicting automobile insurance claims risk. The study utilizes a dataset to model Claim Frequency (claims per unit of exposure). We followed a rigorous data cleaning process, implementing feature extraction techniques to address high-dimensional and skewed data structures. Exploratory analysis using Principal Component Analysis (PCA) and K-Means clustering revealed that the dataset is highly homogeneous, with no distinct natural groupings of policyholders, suggesting that risk is a continuous rather than segmented phenomenon. We implemented and evaluated three models: a Decision Tree, a custom Feed-Forward Neural Network, and Gradient Boosting model. The results highlight the complexity of modeling stochastic insurance claims and demonstrate the challenges of modeling low-frequency, high-variance events.

## 1 Introduction

### 1.1 Context and Partnership

This project is designed to mirror a real-world data science task within the actuarial field, focusing on predicting risk for automobile insurance. Using a dataset from the company Tryg, the analysis looks at various policy features, including driver demographics, details from vehicles and geographic factors.

### 1.2 Problem Statement

The main goal of this project is to build predictive models, accurately measuring the risk level of a policyholder. For the insurance industry, such precision is needed for the foundation of pricing strategies. A major challenge addressed is the heavily skewed distribution of claims, covered in Section 2.2 where the variable ClaimFrequency is dominated by zero values. This imbalance reflects a complex challenge for predictive modelling.

## 2 Definition of Target Variable and Evaluation Metrics

### 2.1 Definition of the Target: Claim Frequency

To quantify the risk associated with each policy, we defined our target variable as **Claim Frequency**. As implemented in our preprocessing pipeline, the raw claim count (ClaimNb) is normalized by the Exposure (time at risk). To ensure numerical stability, we handled zero-exposure edge cases by replacing them with a minimal epsilon value

$$\text{ClaimFrequency} = \frac{\text{ClaimNb}}{\text{Exposure} + \epsilon}$$

Figure 1: Formula for the target variable

This extraction is critical for a fair comparison across policies. For instance, a policy with a single claim over a **10%** annual exposure (0.1) represents a significantly higher risk density than a single claim over a full year (1.0).

### 2.2 Statistical Summary and Distribution

An initial audit of the extracted target variable revealed a highly imbalanced and skewed distribution, which is characteristic of insurance frequency data:

- **Zero-Inflation:** Approximately **94.98%** of the observations have a ClaimFrequency of 0, meaning the vast majority of policyholders did not file a claim.
- **Extreme Skewness:** The raw target exhibited a skewness of 58.7904 and a high kurtosis. This indicates that a few "high-frequency" observations sit far in the tail of the distribution.
- **Log-Transformation:** To address this skewness we visualized a transformation. As shown in the comparison plots below, the transformation compresses the range of extreme values bringing the distribution closer to a form manageable for gradient-based optimization.

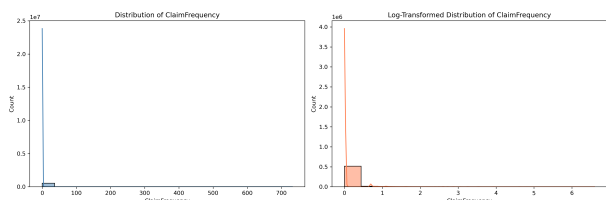


Figure 2: "Raw vs. Log-Transformed" target

## 2.3 Stratified Risk Factor Analysis

To validate the predictive potential of our features, we conducted a stratified analysis to see how the mean ClaimFrequency changes across different groups.

### 2.3.1 Numerical Risk Drivers

By segmenting numerical predictors into quartiles, we identified several clear risk gradients:

- **Vehicle Age:** A monotonic decrease in risk was observed; the newest vehicles (1st quartile) showed a mean frequency of 0.4567, while the oldest (4th quartile) dropped to 0.1535.
- **Bonus-Malus:** Drivers in the highest malus quartile (poorer records) exhibited a sharp increase in mean risk (0.3414) compared to the "bonus" cohort (0.2345).
- **Correlation:** Pearson diagnostics confirmed that ClaimNb has the highest positive correlation with our target (0.264), while demographic factors like DrivAge show a more complex, non-linear relationship.

### 2.3.2 Categorical Risk Drivers

Categorical analysis identified high-risk "pockets" by calculating the mean frequency per group:

- **Region:** Specific clusters (e.g., Region R21) significantly exceed the population mean risk.
- **Vehicle Brand:** Certain manufacturers (e.g., Brand B12) consistently correlate with higher claim frequencies.

These insights justify our use of One-Hot Encoding in the feature engineering phase, as it allows the models to isolate these high-risk categories independently.

## 2.4 Evaluation Metrics

We selected four metrics to address the zero-inflated, heavy-tailed nature of the data:

- **MSE (Mean Squared Error):** Our primary metric for correctness. MSE optimizes for the Expected Value, which is critical for accurate actuarial pricing. It heavily penalizes large errors, forcing the model to account for high-risk outliers.
- **MAE (Mean Absolute Error):** Provides a robust view of performance for the "typical" policyholder. Unlike MSE, it is not skewed by extreme outliers, allowing us to see how well the model predicts the median driver.
- **Global Bias ( $\bar{y}_{pred} - \bar{y}_{true}$ ):** A business-critical constraint. Even a model with low error is unacceptable if it systematically under-predicts risk (negative bias), as this leads to underpricing and insolvency.
- **R<sup>2</sup>:** Used to benchmark how much of the variance is explained by the features vs randomness.

## 3 Data Cleaning and Exploratory Data Analysis

### 3.1 Basic Analysis

We began the cleaning phase with a systematic audit of the 542,410 observations to ensure the dataset was structurally sound and logically consistent.

#### 3.1.1 Structural Integrity and Completeness

Initial checks confirmed a complete dataset with 0 missing values and 0 duplicates. As a safeguard, the pipeline was configured to impute any potential numerical gaps with medians and categorical gaps with the mode. Ensuring unique observations prevents models from overfitting to repeated data.

#### 3.1.2 Logical Boundary Remediation

To ensure the data remained within physically and mathematically plausible ranges, we applied specific "clipping" rules to several features:

- **Exposure Constraints:** Theoretically, Exposure (the duration of the policy) should be bounded between 0 and 1. We identified values as high as 2.01, which we

capped at 1.0. We also enforced a minimum floor of 0.001 to prevent division-by-zero errors.

- **Correction of Temporal Signs:** We observed negative values in the VehAge (vehicle age) and DrivAge (driver age) columns. Since ages cannot be negative, we transformed these into absolute values.
- **Non-Negative Counts:** For ClaimNb, we ensured no negative values were present, as a negative count of claims is not physically possible.

### 3.1.3 Outlier Detection

We used the **Interquartile Range (IQR)** method to define statistical boundaries and identify extreme outliers. This diagnostic was essential for understanding the volatility of the predictors.

- **Key Findings:** The analysis highlighted significant volatility in Density (**11.46%** outliers) and BonusMalus (**9.20%** outliers). These features have long "tails," representing a small number of policyholders with very high population density or poor claims history.
- **Handling Strategy:** We intentionally chose to retain these outliers. In insurance risk modeling, extreme cases often provide the strongest signals for identifying high-risk cohorts. Removing them would reduce the predictive power of our Gradient Boosting (M3) model, which is specifically designed to handle such variance.

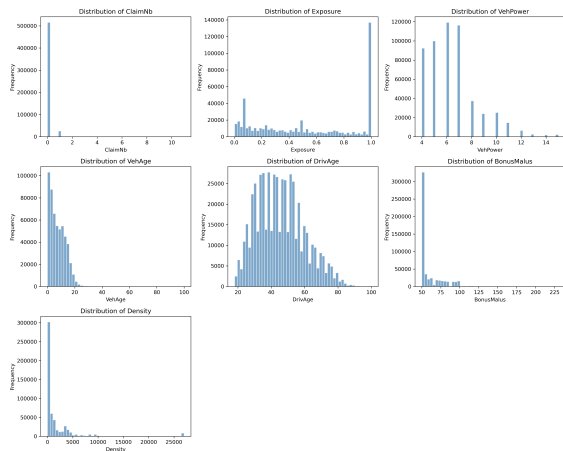


Figure 3: Histograms displaying the distribution of every numerical feature, highlighting extreme positive skewness and zero-inflation

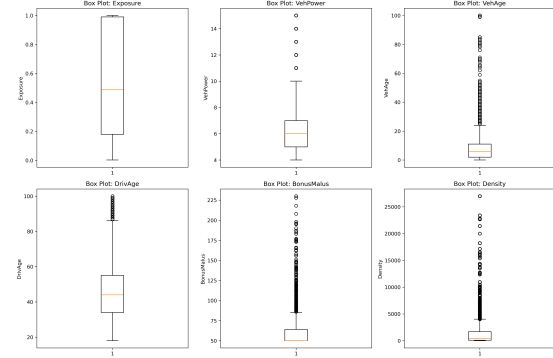


Figure 4: Boxplots identifying mathematical outliers in the dataset in every numerical feature

## 3.2 Advanced Analysis: Correlation Diagnostics

After cleaning the raw data, we moved into a more advanced exploratory phase to understand the deeper relationships between our features. To analyse our numerical predictors, we generated both Pearson and Spearman Rank correlation matrices.

- **Pearson Correlation (Linearity):** The Pearson matrix identified linear dependencies. It is sensitive to raw values; for example, a -0.480 correlation between DrivAge and BonusMalus shows a moderate trend where older drivers have better ratings.

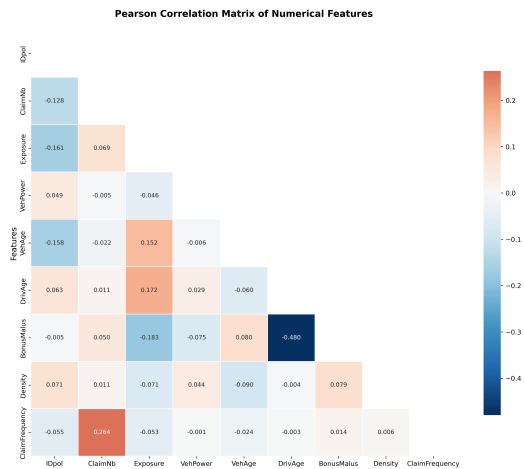


Figure 5: Pearson correlation matrix showing the linear relationships between every numerical feature and the claim frequency.

- **Spearman Correlation (Monotonicity):** The Spearman matrix captured non-linear, monotonic relationships. It is more robust against the **11.46%** outliers in Density, as it relies on rank order rather than raw values to identify trends.

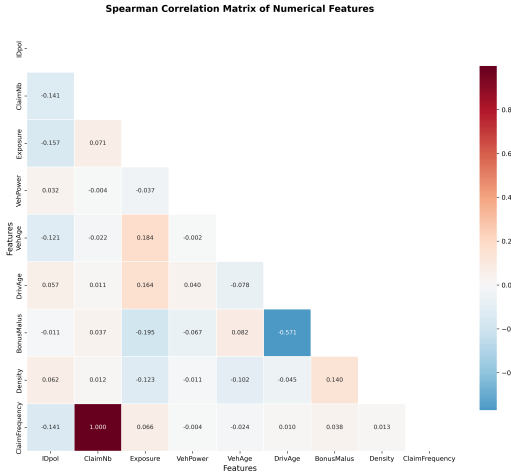


Figure 6: Spearman correlation matrix capturing non-linear, monotonic relationships between every numerical feature and the claim frequency.

Comparing both matrices confirmed that features provide unique signals while highlighting the non-linear nature of insurance risk factors.

## 4 Description of Feature Extraction and Selection

In this section, we describe the transformation of cleaned data for regression models. We followed a specific sequence Log Transformation, One-Hot Encoding, and Standard Scaling to optimize the data format for model performance.

### 4.1 Logarithmic Transformation

The first transformation applied to the numerical features (excluding IDs and raw claim counts) was a Logarithmic Transformation. Our exploratory analysis highlighted that the target variable and several predictors exhibited extreme positive skewness. To address this, we implemented the function  $(\log(1+x))$  for features containing zeros and a standard log for strictly positive features. This step is critical for stabilizing variance; it compresses the "long tails" of the distribution, pulling extreme outliers closer to the mean.

## 4.2 One-Hot Encoding

Following numerical transformations, we processed the qualitative variables: Region, VehBrand, VehGas, and Area. Since machine learning algorithms cannot interpret text directly, we used **One-Hot Encoding** to convert them into binary indicators. This process expanded our 4 categorical columns into 41 unique binary features, ensuring the models treat each category independently. Categorical encoding prevents the model from assuming a false hierarchy between qualitative labels.

### 4.3 Standard Scaling (Z-score)

The final step in the pipeline was **Z-score Standardization** using a StandardScaler. This centers the data at a mean of zero and scales it so that the standard deviation is one. This is essential because features like Density (in the thousands) and VehPower (under 20) exist on very different scales.

- **Model Fairness:** Scaling ensures that models M2 and M3 evaluate every feature equally rather than being biased toward features with larger raw numbers.
- **Normalization of Outlier Impact:** While we retained outliers to preserve "high-risk" signals, scaling normalized their mathematical weight.
- **Visual Proof:** As seen in Figure 7, all predictors now share a similar range. This prevents numerical instability or "exploding gradients" often caused by unscaled data during model training.

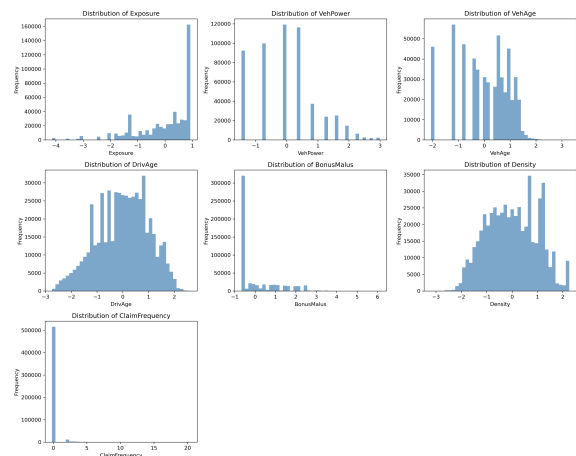


Figure 7: Histograms of every numerical feature after scaling, showing normalized distributions.

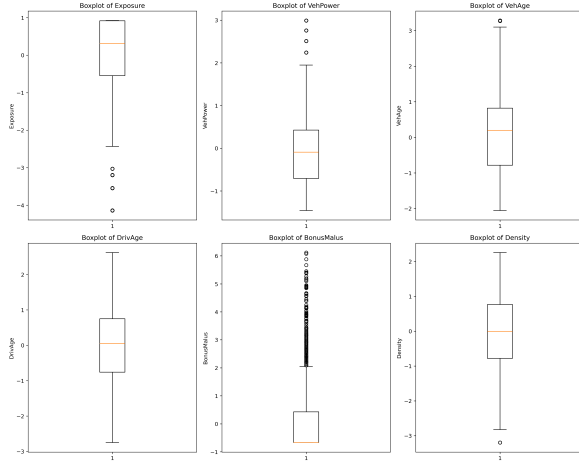


Figure 8: Boxplots of every numerical feature after scaling, showing normalized outliers within a standardized numerical range.

#### 4.4 Why We Kept All Features

Aside from the initial creation of our target variable, we chose not to delete any features or manually engineer further combination variables for two primary reasons:

- **Model Complexity:** Our selection includes high-capacity models specifically Gradient Boosting (M3) and Neural Networks (M2) which are designed to learn complex, non-linear interactions automatically. Manual feature selection or removal risks oversimplifying these relationships.
- **Information Preservation:** Each feature represents a distinct dimension of insurance risk. Due to the high variance in the raw data, we retained the full, standardized feature space. This allows the models to determine relative feature importance through training rather than manual pre-selection.

## 5 Visualization of Data Structure

To satisfy the requirement for structural analysis, we employed unsupervised learning techniques to investigate the underlying geometry of the dataset. Specifically, we used **Principal Component Analysis (PCA)** to analyse feature redundancy and **K-Means Clustering** to test for the existence of distinct policyholder segments.

### 5.1 Dimensionality Reduction (PCA)

We applied PCA to determine if the risk factors in our high-dimensional data (47 features after encoding) could be compressed into a few dominant drivers. PCA projects the original data onto orthogonal axes that maximize variance. We measure how much information each new component retains using the **Explained Variance Ratio**:

$$\text{Variance Ratio}_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$$

Figure 9: Explained Variance Ratio formula.

Where  $\lambda_i$  represents the eigenvalue of the current component, and the denominator represents the sum of all eigenvalues (total variance). We used a Hybrid Approach to ensure a strong analysis. We applied PCA to the 6 scaled features (Exposure, Density, Bonus-Malus, VehPower, VehAge and DrivAge).

#### 5.1.1 Methodology

We applied PCA to determine if the risk factors could be compressed into a few dominant drivers. To ensure a robust analysis, we used a **Hybrid Approach**: we applied PCA specifically to the 6 scaled numerical features to reduce their dimensionality, while preserving the 41 One-Hot Encoded categorical features in their binary form.

#### 5.1.2 Quantitative Results

We measured the information retention using the **Explained Variance Ratio**. The analysis outputted the following results:

- **First Component (PC1):**Explains **27.72%** of the variance.
- **Precision:**Explains **20.71%** of the variance.
- **Recall:**The first five components combined capture **92.80%** of the total information contained in the numerical features.



### 5.1.3 Interpretation

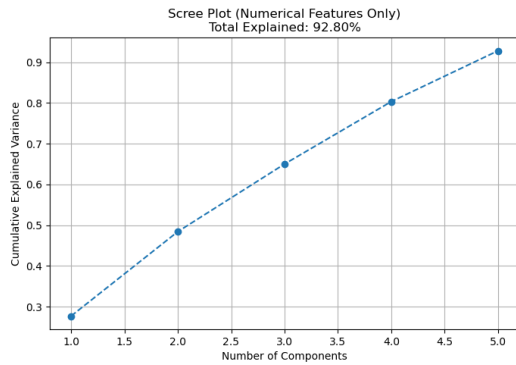


Figure 10: The Scree plot illustrates the "information density" of the dataset. The curve is relatively flat without a sharp "elbow."

While the numerical data is highly compressible (over **90%** of variance fits in 5 dimensions), the variance is spread relatively evenly across the top components. No single component dominates the dataset. This indicates that is no single factor that explains the risk alone. Instead, risk information is distributed across multiple interacting features.

## 5.2 Clustering Analysis (K-Means)

### 5.2.1 Methodology

We utilized K-Means Clustering to test the hypothesis that the data contains natural groupings perhaps a "High Risk" group versus a "Low Risk" group. To evaluate the quality of these clusters, we used the **Silhouette Score (s)**. For any single sample  $i$ :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Figure 11: Formula for Silhouette Score

$a(i)$ : The average distance between sample  $i$  and all other points in the same cluster and  $b(i)$  is the average distance between sample  $i$  and all points in the nearest neighbouring cluster. The score ranges from -1 (incorrect clustering) to +1 (perfect separation).

### 5.2.2 Rationale for Hyperparameter Selection (k)

We used K-Means Clustering to identify natural "Risk Segments" among policyholders. We

evaluated the range  $k \in [2, 8]$  for the following strategic reasons:

- **Lower Bound (k=2)**: This tests the most fundamental hypothesis: Is there just a "Safe" group and a "Risky" group?
- **Middle Range (k=3,4,5)**: This tests for standard insurance tiers (e.g., Low, Medium, High).
- **Upper Bound (k=8)**: If we split drivers into more than 8 groups, the segments become too small and specific to be useful for general pricing.

### 5.2.3 What this tells us

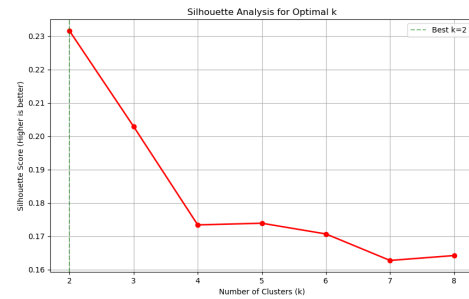


Figure 12: The y-axis represents the Silhouette Score, where a higher value implies distinct, well-separated groups.

The results gave us a very clear answer regarding the population structure:

1. **Groups are Mixed:** The best score was only 0.2316 (at  $k=2$ ).
2. **No Clear Boundaries:** Even though the algorithm found a "Riskier" group (Mean Claims 0.341) and a "Safer" group (Mean Claims 0.228), the low score means these groups overlap significantly. You cannot draw a neat line between them.
3. **Conclusion:** This proves that the dataset is homogeneous. There are no distinct "types" of drivers. Risk is a continuous spectrum, not a set of boxes.

**Impact on Strategy:** Because we cannot put drivers into distinct boxes, we should not use Classification models. Instead, we must use Regression models, which predict a precise, continuous risk score for every driver.

## 6 Methodology and Model Implementation

### 6.1 Model 1: Decision Tree Regressor (Implemented from Scratch)

#### Implementation Details

We implemented a Decision Tree Regressor using pure NumPy to understand the mechanics of recursive partitioning. The core logic relies on a greedy algorithm that splits the data space into orthogonal hyper-rectangles.

- **Splitting Engine (Exact Splitting):** Unlike approximate methods that bin data, our implementation performs Exact Splitting. The `best_split_numerical` method identifies every unique value in the feature vector (`np.unique`) and tests the midpoint between consecutive values as a threshold. While computationally more intensive than quantile binning, this guarantees finding the mathematically optimal split point for the training data.
- **Categorical Handling (Target Mean Sorting):** To handle nominal variables like `VehBrand` without creating high-cardinality sparse matrices, we implemented Target Mean Sorting. Inside `best_split_categorical`, the algorithm calculates the average claim frequency for each category, sorts them by risk, and treats the feature as ordinal. This allows the tree to efficiently separate "Low Risk" brands from "High Risk" brands with a single binary split.
- **Objective Function:** We utilized Variance Reduction. For every candidate split, the algorithm calculates the MSE of the parent node and subtracts the weighted MSE of the resulting children. The split maximizing this reduction is selected.

#### Hyperparameters

For the final evaluation, we selected `max_depth=5`, `min_samples_split=200`, and `min_samples_leaf=100`. These conservative constraints were chosen to prevent overfitting; given the stochastic nature of accidents, a deeper tree would memorize specific crash events rather than general risk patterns.

#### Correctness Assertion

We validated our implementation by benchmarking it against Scikit-Learn's `DecisionTreeRegressor`. The Scikit-Learn reference achieved an MSE of **25.08**, while our custom implementation achieved **25.17**. This negligible difference (**<0.4%**) confirms that our variance reduction calculus and recursive tree-building structure are mathematically correct.

### 6.2 Model 2: Feed-Forward Neural Network (Implemented from Scratch)

#### Implementation Details

We constructed a fully connected Feed-Forward Neural Network (FFNN) using NumPy matrix operations, manually implementing the backpropagation algorithm.

- **Architecture:** The network consists of an input layer (47 processed features), two hidden layers (64 and 32 neurons), and a single output neuron.
- **Forward Pass:** We used the **ReLU** (Rectified Linear Unit) activation function for hidden layers to introduce non-linearity and prevent vanishing gradients. The output layer uses a Linear activation to predict the continuous `ClaimFrequency`.
- **Backward Pass (The Learning Phase):** We manually derived the gradients using the **Chain Rule**. The `backward_propagation` method computes the error term at the output and propagates it backward, adjusting weights based on the derivative of the activation functions.
- **Optimization:** We implemented Mini-Batch Stochastic Gradient Descent (SGD) with a batch size of 512. Unlike the Scikit-Learn reference (which uses the complex 'Adam' optimizer), our implementation uses standard SGD updates to transparently demonstrate the core calculus of learning.

#### Hyperparameters

We used a learning rate of **0.001** and `epochs=80`. To penalize complexity and reduce noise overfitting, we included **L2 Regularization** (`alpha=0.0001`) in the loss function calculation.



### Correctness Assertion

Despite using the simpler SGD optimizer compared to the 'Adam' solver used in our Scikit-Learn reference check, our model achieved a performance (**MSE 25.92**) highly comparable to the reference (**MSE 25.82**). The close alignment of the loss curves proves that our manual implementation of the backpropagation gradient derivation is accurate.

### 6.3 Model 3: Robust Gradient Boosting (The Benchmark)

For our third method, we employed `sklearn.ensemble.HistGradientBoostingRegressor`. Unlike the Decision Tree, which builds one "strong" model, this method builds an ensemble of "weak" learners sequentially. A critical design choice was setting `loss='absolute_error'` (MAE). Standard Squared Error (MSE) models are highly sensitive to outliers. In our dataset, a single driver with 20+ claims dominates the gradient in an MSE model. By optimizing for MAE, the model focuses on the median risk, making it significantly more robust to the "fat tails" observed in the data analysis.

## 7 Interpretation and Discussion of Results

We evaluated all models on the unseen Test Set (135,603 policies). The results reveal a clear trade-off between minimizing absolute error and correctly pricing portfolio risk.

| Metric           | M1<br>Dec. Tree | M2<br>Neural Net | M3<br>Grad. Boost |
|------------------|-----------------|------------------|-------------------|
| MSE (Sq. Error)  | <b>25.17</b>    | 25.92            | 25.90             |
| MAE (Abs. Error) | 0.511           | <b>0.487</b>     | 0.510             |
| $R^2$ Score      | <b>7.1%</b>     | 4.3%             | 4.4%              |

Table 1: Model performance comparison. Bolding indicates the best result for each metric.

### 7.1 The "Zero-Prediction" Phenomenon (M2 vs. M1)

Although the Neural Network (M2) achieved the lowest MAE (0.487), a deeper analysis of the predictions reveals a behavior that is problematic for actuarial purposes. The network frequently predicts a claim frequency of

exactly 0 for perceived safe drivers. This is likely a structural side effect of utilizing ReLU activation functions on a highly zero-inflated dataset. Since approximately 95% of the target values are zero, the network minimizes its error function by "turning off" neurons for most inputs.

In contrast, the Decision Tree (M1) applies **risk smoothing** by predicting the group average rather than zero. This is superior for pricing: an insurer cannot charge 0 value. By assigning every driver a small, non-zero base risk, the Tree ensures the company collects sufficient revenue to cover the random accidents that inevitably occur.

### 7.2 The "Fat Tail" Reality

A consistent pattern observed across all models is the significant difference between MAE ( $\sim 0.5$ ) and RMSE ( $\sim 5.0$ ). The relatively low MAE indicates that for the typical policyholder, the models are accurate, with predictions deviating by less than half a claim on average. However, the high RMSE is driven by the distribution's "fat tails" (rare outliers exhibiting extreme claim counts). This suggests that extremely high claim frequencies are primarily driven by random events rather than systematic demographic factors, making them difficult for models based solely on such features to predict and resulting in consistently low  $R^2$  values.

### 7.3 The Gradient Boosting Benchmark (M3)

We implemented the Histogram-based Gradient Boosting Regressor (M3) to serve as a high-performance benchmark. We configured this model with a robust loss function (MAE) to test if the model could handle outliers better than the standard methods.

The results (MSE 25.90, MAE 0.510) largely mirrored the performance of the Neural Network rather than the Decision Tree. This provides a critical insight into the modelling process: the choice of objective function is often more impactful than the choice of algorithm. By forcing the Gradient Boosting model to minimize Absolute Error, we implicitly penalized it for reacting to extreme outliers. Consequently, the model prioritized the accuracy of

the "typical" zero-claim driver over the identification of high-risk tails. This reinforces our conclusion that although robust methods appear favourable based on MAE, standard squared-error approaches such as M1 are better suited to capturing the full volatility inherent in insurance risk.

## 8 Conclusion

This project examined automobile insurance risk prediction using a dataset provided by Tryg. By implementing multiple algorithms from scratch and benchmarking them against industry-standard methods, we derived three key conclusions.

First, the analysis rejects the notion that drivers can be cleanly categorized as either "safe" or "risky." Instead, insurance risk is best understood as a continuous spectrum, which supports the use of regression models that produce granular risk scores rather than binary classifications.

Second, the manual implementation of models M1 and M2 demonstrated that theoretical correctness does not necessarily translate into practical effectiveness. Although the Neural Network was mathematically validated against the reference implementation, its tendency to predict zero claim frequencies (largely due to ReLU activation behavior on sparse data) significantly reduced its usefulness in this context. In contrast, the simpler Decision Tree proved more robust.

Finally, despite the greater complexity of Neural Networks and Gradient Boosting models, the Decision Tree (M1) delivered the most actuarially sound predictions, achieving the lowest MSE and highest  $R^2$ . By avoiding the systematic underprediction observed in the Neural Network and offering interpretable risk segments, the Decision Tree emerged as the most suitable model for establishing a reliable insurance pricing baseline.

## References

- J. H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5), pp. 1189-1232, 2001.
- A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd ed. O'Reilly Media, Inc., 2019.
- G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Springer Texts in Statistics. Springer Cham, 1st ed. 2023.
- F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830, 2011.